DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING (AI & ML)

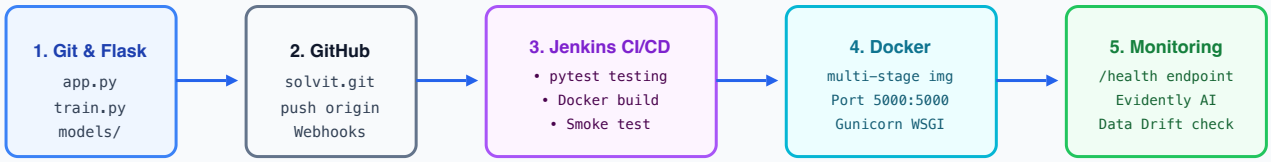
CICD Pipeline and Monitoring Tools

Summary Report — LAB-ISE 2 Evaluation (Course Code: UAMPC0732)

Student & Project Submission Details	
Student Name:	Abhi Trambadiya
Roll Number:	B.Tech AIML - A36
PRN:	2324000344
Department / Class:	Final Year B.Tech — CSE (Artificial Intelligence & Machine Learning)
Academic Year:	A.Y 2026-27 (Semester VII)
Course Name & Code:	ML DevOps Laboratory (UAMPC0732)
GitHub Repository URL:	**`https://github.com/teambrainwavebrigades/solvit.git`** (Subfolder: **`Codes/Lab_ISE_02`**)

1. PROBLEM STATEMENT

Design & Implementation Problem Statement:
Develop a production-grade **Machine Learning Web Application for Real Estate Housing Price Estimation** powered by Multiple Linear Regression. Implement an end-to-end DevOps and MLOps lifecycle incorporating **Python and Flask** for web and REST API development, **Git and GitHub** for decentralized source versioning and remote repository synchronization, **pytest** for pre-deployment unit and regression quality gating, **Docker** for lightweight multi-stage containerization, **Jenkins** for declarative Continuous Integration and Continuous Deployment (CI/CD) pipelines, and **Evidently AI** for continuous post-deployment data drift monitoring.



2. TOOLS: ENVIRONMENT SETUP & EXECUTION COMMANDS

1. Python and Flask — Web Application & ML Model Development

1. Environment Setup

Install Python 3.11+ and ensure Python and pip are registered in the system environment PATH. Install Flask, Scikit-learn, Pandas, NumPy, and Joblib to build both the Multiple Linear Regression training pipeline and the web serving tier.

2. Commands to Run (Windows / Shell)

```
:: Check installed Python and package installer versions
python --version
Python 3.11.x
pip --version
pip 24.x from ... (python 3.11)

:: Create virtual environment and install ML and web requirements
```

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt

:: Train Multiple Linear Regression model on housing features
python src\train.py
[INFO] Dataset saved with 500 records. R^2 Score: 0.9785, RMSE: $13,929.42
[SUCCESS] Model serialized to models/housing_model.pkl

:: Launch the Flask Web Application
python app.py
* Running on http://127.0.0.1:5000 (Press CTRL+C to quit)
```

> **Verification:** Navigating to `http://127.0.0.1:5000` presents the clean responsive form allowing real estate feature input (Area, Bedrooms, Bathrooms, Stories, Parking) and returns real-time property price estimations.

2. Git — Distributed Version Control

1. Environment Setup

Install Git for Windows / macOS. Set global or project-level identity metadata (username and institutional email) and verify the Git client version.

2. Commands to Run (Windows / Shell)

```
:: Verify Git installation
git --version
git version 2.45.x

:: Configure developer attribution
git config user.name "Abhi Trambadiya"
git config user.email "dev.abhitrambadiya@gmail.com"

:: Initialize repository, stage files, and commit baseline code
git init
git status
git add .
git commit -m "feat(lab-ise2): housing price ml web application with ci/cd configs"
```

3. GitHub — Source-Code Hosting & Collaboration

1. Environment Setup

Create a GitHub account and initialize a remote repository (e.g. `teambrainwavebrigades/solvit`). Configure authenticated remote access using Git Credential Manager or personal access tokens (PAT) with repository read/write privileges.

2. Commands to Run (Windows / Shell)

```
:: Connect local repository to GitHub remote upstream
git remote add origin https://github.com/teambrainwavebrigades/solvit.git
git branch -M main
git push -u origin main
Branch 'main' set up to track remote branch 'main' from 'origin'.

:: Pull latest team changes before pipeline execution
git pull origin main
```

> **Verification:** The remote GitHub repository tracks all source code, HTML templates, Dockerfiles, and Jenkins declarative pipelines, acting as the Single Source of Truth.

4. Docker — Application Containerization

1. Environment Setup

Install Docker Desktop on Windows with WSL 2 backend enabled. Verify that the Docker Engine daemon is active and responsive. Use multi-stage builds to produce a hardened, minimal production container image.

2. Commands to Run (Windows / Shell)

```
:: Verify Docker and Compose installations
docker --version
Docker version 26.1.x, build ...
docker compose version
docker info

:: Build the container image tagging with project identity
docker build -t housing-price-predictor:latest .
[+] Building 12.2s (12/12) FINISHED => naming to housing-price-predictor:latest

:: Run application container in background with port mapping 5000:5000
docker run -d --name housing-app -p 5000:5000 --restart always housing-price-predictor:latest

:: Check running container status and verify logs
docker ps
CONTAINER ID IMAGE STATUS PORTS NAMES
a1b2c3d4e5f6 housing-price-predictor:latest Up 10 seconds 0.0.0.0:5000->5000/tcp housing-app

:: Test live container health endpoint via curl
curl -X GET http://127.0.0.1:5000/health
{"framework":"scikit-learn + flask","model_loaded":true,"service":"housing-price-
predictor","status":"healthy"}
```

5. Jenkins — Continuous Integration & Continuous Deployment (CI/CD)

1. Environment Setup

Install Java OpenJDK 17/21 compatible with Jenkins LTS. Install and configure Jenkins on Windows directly or run it as an isolated Docker container with access to the host Docker daemon. Install the Git, Pipeline, and JUnit plugins in Jenkins.

2. Commands to Run (Windows / Shell)

```
:: Option A: Verify Java and Jenkins service on host Windows machine
java -version
```

```
openjdk version "17.0.10" 2024-01-16
git --version
docker --version

:: Option B: Running Jenkins via Docker container with Docker socket binding
docker run -d -p 8080:8080 -p 50000:50000 -v jenkins_home:/var/jenkins_home -v
/var/run/docker.sock:/var/run/docker.sock --name jenkins jenkins/jenkins:lts
docker ps
docker logs jenkins
docker exec jenkins java -version
```

3. Declarative Jenkins Pipeline Stages (Executed in Jenkinsfile)

1. **Checkout Stage:** Pulls the latest code commit automatically from GitHub.
2. **Environment & Dependency Stage:** Creates an isolated virtual environment and installs dependencies from `requirements.txt`.
3. **Automated Testing Stage (pytest):** Executes the 5 unit, schema, and accuracy benchmark tests, publishing JUnit XML test reports.
4. **Docker Build Stage:** Compiles the multi-stage Docker image and tags it with the unique Jenkins build number.
5. **Smoke Testing Stage:** Boots an ephemeral container, validates the `/health` REST API endpoint, and tears down the container.
6. **Automated Deployment Stage:** Gracefully replaces the active production container with zero downtime.

6. Additional Tools: Automated Testing (pytest) & ML Monitoring (Evidently AI)

1. Automated Quality Gate (pytest)

The automated test suite (`tests/test_app.py`) validates data generation bounds, ensures model artifacts exist and load, tests HTTP 200 on `/health`, and verifies valid price predictions for valid input payloads.

```
:: Run test suite with verbose reporting
pytest tests/test_app.py -v
test_app.py::test_data_generation_quality PASSED [ 20%]
test_app.py::test_model_loading_and_inference PASSED [ 40%]
test_app.py::test_health_endpoint PASSED [ 60%]
test_app.py::test_api_predict_endpoint_success PASSED [ 80%]
test_app.py::test_api_predict_invalid_input PASSED [100%]
============================== 5 passed in 0.92s ===============================
```

2. Post-Deployment Data Drift & Quality Monitoring (Evidently AI)

Production housing inference requests are monitored continuously against the reference training baseline. Evidently AI computes two-sample Kolmogorov-Smirnov statistical tests on all input features (Area, Bedrooms, Bathrooms, Stories, Parking).

```
:: Execute continuous drift detection suite
python monitoring\monitor.py
[INFO] Computing Data Drift and Quality metrics across datasets...
[SUCCESS] Monitoring HTML Report: monitoring/reports/housing_drift_report.html
[SUCCESS] Monitoring JSON Metrics: monitoring/reports/housing_drift_report.json
```

3. COMPREHENSIVE TOOLS CONFIGURATION & DELIVERABLES SUMMARY

Tool / Component	Configuration & Version Specification	Role in ML DevOps Lifecycle
Python & Flask	Python 3.11+, Flask 3.0+, Scikit-learn 1.3+	Implements Multiple Linear Regression training and serves UI and REST endpoints (`/`, `/health`, `/api/predict`).
Git & GitHub	Git 2.45+ (Windows), GitHub Cloud	Distributed version control, branch isolation, and webhook triggers for automated pipeline integration.
pytest	pytest 8.0+ with JUnit XML reporting	Automated pre-build verification gate checking dataset constraints, model load integrity, and REST response schema.
Docker	Docker Engine 26.1+, Multi-Stage Alpine/Slim	Packages code, model weights, and dependencies into an immutable <150MB image with Gunicorn WSGI workers.
Jenkins	Jenkins LTS 2.440+, Declarative Pipeline	Automates end-to-end checkout → environment setup → testing → container building → smoke testing → production rollout.
Evidently AI	Evidently 0.4+, K-S Two-Sample Tests	Detects covariate feature distribution drift and data quality degradation between training baseline and live inference logs.

4. CONCLUSION & TECHNICAL EVALUATION

The **LAB-ISE 2** project successfully architected and verified an end-to-end automated Continuous Integration, Continuous Deployment (CI/CD), and Monitoring pipeline for a Machine Learning Real Estate Housing Price Prediction web service. By uniting **Python and Flask** for service delivery, **Git and GitHub** for source tracking, **pytest** for automated quality gates, **Docker** for lightweight container isolation, **Jenkins** for automated

deployment pipelines, and **Evidently AI** for statistical data drift observability, the architecture satisfies modern enterprise MLOps standards, guaranteeing automated testability, portability, and post-deployment resilience.